Flutter Clean Architecture

Universal API Integration Guide by huthaifa Ma'in [linkedIn]

Dio · BLoC / Cubit · Clean Architecture

Introduction

This guide provides a repeatable, endpoint-agnostic blueprint for connecting any Flutter feature to any backend endpoint using **Dio for networking**, **Cubit for state management**, and **Clean Architecture for code organisation.**

Every time you receive a new API document from your backend team, follow the **same 10-step path** described here. The only things that change between features are the **endpoint URL**, the **field names**, and the **Dart class names** — **the structure never changes.**

How to Read an API Document

When you open a backend API document, identify and note these five things before writing a single line of code:

1. **The Method (GET / POST / PUT / DELETE)** — tells you how to communicate. POST sends data to create or trigger something. GET retrieves data without a body. PUT updates an existing resource. DELETE removes a resource.
2. **The Path (e.g. /api/products)** — the exact path you append to your base URL. Copy it character-for-character.
3. **The Request Body** — what the backend expects you to send. For GET and DELETE requests this is usually empty; for POST and PUT it is a JSON object. The keys you find here are the fields your UI must collect.
4. **The Response Body (Success)** — what the backend returns on success. The keys and nested objects here define exactly what your Entity and Model must contain.
5. **The Error Codes** — the HTTP status codes listed as failures (e.g. 400, 401, 404, 422). These are the exceptions your Repository must catch and convert into friendly UI messages.

> 💡 **TIP:** The response body is your single source of truth for Step 2 (Entity) and Step 3 (Model). Every field you need in your UI must be declared there.

The Data Flow Roadmap

Every feature you build follows the same path from the deepest configuration layer outward to the user's screen:

```
1.  Endpoint URL
          ↓
2.  Entity (Pure Dart blueprint)
          ↓
3.  Model (JSON parser – extends Entity)
          ↓
4.  Remote Data Source (Dio call)
          ↓
5.  Repository Interface (domain contract)
          ↓
6.  Repository Implementation (error handling)
          ↓
7.  UseCase (single-action bridge)
          ↓
8.  Cubit + States (state management)
          ↓
9.  Dependency Injection (wiring with get_it)
          ↓
10. UI Screen (BlocProvider + BlocConsumer)
```

Start at Step 1 and drive outward. Never skip a layer, and never let a layer reach past its immediate neighbour.

Step 1 Add the Endpoint URL

Store every endpoint path in one central constants file. This ensures that if a path ever changes, you update it in a single place.

File: lib/core/api/end_points.dart

Feature Example

```dart
class EndPoint {
  // Platform-aware base URL
  static String get baseUrl {
    if (Platform.isAndroid) {
      return 'http://10.0.2.2:8000'; // Android emulator
    } else if (Platform.isIOS) {
      return 'http://localhost:8000'; // iOS simulator
    } else {
      return 'https://your-production-api.com';
    }
  }

  // – Auth ----------------------------------------
  static String login    = '/auth/login';
  static String register = '/auth/register';
  static String logout   = '/auth/logout';
```

```
  // - Feature Example ----------------------------
  // static String featureName = '/api/path';
}
```

📌 **NOTE:** One line per endpoint. When your backend team sends you a new path, add it here first before touching any other file.

Step 2 Create the Entity

An Entity is a pure Dart class that holds only the fields your app actually uses. It contains zero JSON logic and zero framework dependencies — it belongs entirely to the domain layer.

Folder: lib/features/[feature]/domain/entities/

Siblings vs. Nested — Architectural Distinction

Sibling Entities

Sibling entities are objects or fields that **exist at the exact same level** in the API JSON response — for example, token, user, and company living side-by-side inside a data object. Do not nest them inside each other. Create a Parent or Wrapper Entity (such as AuthEntity) that holds all the siblings as separate fields.

Nested Entities

A nested entity is **a JSON object that lives entirely inside another JSON object** — for example, a company object returned inside the user object. Create a separate pure Dart Entity for the nested object and reference it as a typed field inside the parent entity.

Data Type Notes

- **Common Types:** Use exact primitive Dart types: String, int, double, and bool.
- **Lists:** For arrays of items, use List with the specific type (e.g., List<String> or List<ItemEntity>).
- **Nullability:** If the API might omit a field or return null, add a question mark to the type (e.g., String?). Do not use the required keyword in the constructor for these nullable fields.
- **Custom Object Types:** For nested JSON objects, the data type should be the corresponding pure Dart Entity class (e.g., final CompanyEntity company;).

Universal Template — Standard Entity (Flat / Siblings)

Universal Template

```
class StandardEntity {
  final Type  requiredField;
  final Type? optionalField;

  const StandardEntity({
    required this.requiredField,
    this.optionalField,
  });
}
```

Universal Template — Nested Entity

When an object contains another object, define the child first, then reference it in the parent.

Universal Template

```
// 1. Define the Nested (Child) Entity
class ChildEntity {
  final String childField;

  const ChildEntity({
    required this.childField,
  });
}

// 2. Define the Parent Entity
class ParentEntity {
  final int       id;
  final ChildEntity nestedObject; // The type is the ChildEntity class

  const ParentEntity({
    required this.id,
    required this.nestedObject,
  });
}
```

Feature Example — Flat Entity

Feature Example

```
// ProductEntity – declare only the fields your UI needs.
// Source: the 'success' response body of your API doc.
class ProductEntity {
  final int    id;
  final String name;
  final double price;
  final String category;

  const ProductEntity({
    required this.id,
    required this.name,
    required this.price,
    required this.category,
  });
}
```

Feature Example — Nested Entity (User with Company)

Feature Example

```
// company_entity.dart
class CompanyEntity {
  final int    id;
  final String name;
  const CompanyEntity({ required this.id, required this.name });
```

```
}

// user_entity.dart
import 'company_entity.dart';

class UserEntity {
  final String        token;
  final int           id;
  final String        name;
  final CompanyEntity company; // <-- typed nested entity

  const UserEntity({
    required this.token,
    required this.id,
    required this.name,
    required this.company,
  });
}
```

💡 **TIP:** Look at your API's success response body. Every JSON key you need in your UI becomes a field here. Fields you will never display can be omitted.

Step 3 Create the Model

A Model extends its matching Entity and adds a fromJson factory constructor. It acts as a translator, converting the raw JSON map from the backend into a typed Dart object.

Folder: lib/features/[feature]/data/models/

Quick Rules

- **Extension:** Every Model must **extend** its matching Entity.
- **Factory:** Every Model needs a **factory .fromJson() constructor**.
- **Exact Matches:** The JSON keys inside fromJson must be copied character-for-character from your API doc. A single typo causes a null value or a runtime crash.

Universal Template — Standard Model

Universal Template

```
import '../../domain/entities/your_entity.dart';

class YourModel extends YourEntity {
  const YourModel({
    required super.requiredField,
    super.optionalField, // Do not use 'required' for nullable fields
  });

  // Keys MUST exactly match the JSON keys in the API doc.
```

```
  factory YourModel.fromJson(Map<String, dynamic> json) {
    return YourModel(
      requiredField: json['exact_json_key'],
      optionalField: json['optional_json_key'],
    );
  }
}
```

Feature Example — Flat Model

Feature Example

```
import '../../domain/entities/product_entity.dart';

class ProductModel extends ProductEntity {
  const ProductModel({
    required super.id,
    required super.name,
    required super.price,
    required super.category,
  });

  // Keys MUST exactly match the JSON keys in the API doc.
  factory ProductModel.fromJson(Map<String, dynamic> json) {
    return ProductModel(
      id:       json['id'],
      name:     json['name'],
      price:    (json['price'] as num).toDouble(),
      category: json['category'],
    );
  }
}
```

Parsing Sibling / Nested Objects — The Parent Model Delegator

When an API returns multiple distinct objects at the same level (siblings) inside the response, use a **Parent Model to group them**. The Parent Model's fromJson factory acts as a traffic director: it extracts primitive fields directly, and for any complex objects it extracts that specific JSON map and delegates to the corresponding child model's fromJson factory.

Universal Template — Parent Model with Siblings

Universal Template

```
import 'child_a_model.dart';
import 'child_b_model.dart';

class ParentModel extends ParentEntity {
  const ParentModel({
    required super.primitiveField,
    required super.childObjectA,
    required super.childObjectB,
  });

  factory ParentModel.fromJson(Map<String, dynamic> json) {
```

```
    return ParentModel(
      // 1. Extract primitive types directly
      primitiveField: json['primitive_key'],

      // 2. Delegate specific object maps to their respective Model factories
      childObjectA: ChildModelA.fromJson(json['object_a_key']),
      childObjectB: ChildModelB.fromJson(json['object_b_key']),
    );
  }
}
```

Feature Example — Nested Model (User with Company)

Feature Example

```
import '../../domain/entities/user_entity.dart';
import 'company_model.dart';

class UserModel extends UserEntity {
  const UserModel({
    required super.token,
    required super.id,
    required super.name,
    required super.company,
  });

  factory UserModel.fromJson(
    Map<String, dynamic> userJson,
    String               token,
    Map<String, dynamic> companyJson,
  ) {
    return UserModel(
      token:   token,
      id:      userJson['id'],
      name:    userJson['name'],
      company: CompanyModel.fromJson(companyJson),
    );
  }
}
```

Feature Example — Model with List Support

Feature Example

```
// Inside ProductModel:
static List<ProductModel> fromJsonList(List<dynamic> jsonList) {
  return jsonList
    .map((item) => ProductModel.fromJson(item as Map<String, dynamic>))
    .toList();
}
```

💡 **TIP:** The JSON keys inside fromJson must be copied character-for-character from your API doc. A single typo causes a null value or a runtime crash.

📌 **NOTE: Defensive Parsing in fromJson —** You may add if statements and null checks inside fromJson when the backend cannot guarantee a field's type or presence. This is valid and encouraged in two situations: (1) a field might arrive as either a String or an int (e.g. id can be "5" or 5), in which case use a type check (json['id'] is String ? int.tryParse(...) : json['id']); and (2) a field might be absent or null in some API responses, in which case use the null-coalescing operator (??) to supply a safe fallback (e.g. json['content'] ?? "No terms available."). Fallback values such as -1 or "0.0.0" make it immediately visible in the UI that a field was missing. The standard template intentionally omits these checks to stay concise; add them whenever your specific API requires it.

Step 4 Write the Remote Data Source

The Remote Data Source is the only place in your app that talks to the network. It has three distinct responsibilities: call Dio (GET/POST/PUT/DELETE), pass the correct request body, and return a typed Model mapped from the raw JSON response. You must define an **abstract class (the interface)** and an **implementation class** that uses implements.

Folder: lib/features/[feature]/data/datasources/

Universal Template — POST (Sends a body, returns an object)

Universal Template

```
import '../../../../core/api/end_points.dart';
import '../../../../core/api/dio_client.dart';
import '../models/your_model.dart';

// 1. The Interface
abstract class FeatureRemoteDataSource {
  Future<YourModel> performAction(String inputA, String inputB);
}

// 2. The Implementation
class FeatureRemoteDataSourceImpl implements FeatureRemoteDataSource {
  final DioClient dioConsumer;
  FeatureRemoteDataSourceImpl({required this.dioConsumer});

  @override
  Future<YourModel> performAction(String inputA, String inputB) async {
    final response = await dioConsumer.post(
      EndPoint.yourEndpointPath,
      data: {
        'api_key_1': inputA, // Keys must match the API doc request body exactly
        'api_key_2': inputB,
      },
    );
    // Adjust the path to match your API's response structure
    final data = response.data['data'];
    return YourModel.fromJson(data);
  }
}
```

Universal Template — GET (Single object)

Universal Template

```
  @override
  Future<YourModel> getSingleItem(int id) async {
    final response = await dioConsumer.get('${EndPoint.yourEndpoint}/$id');
    final data = response.data['data'];
    return YourModel.fromJson(data);
  }
```

Universal Template — GET (List of objects)

Universal Template

```
  @override
  Future<List<YourModel>> getItems() async {
    final response = await dioConsumer.get(EndPoint.yourEndpoint);
    final List<dynamic> list = response.data['data'];
    return YourModel.fromJsonList(list);
  }
```

Universal Template — PUT (Updates an existing resource)

Universal Template

```
  @override
  Future<YourModel> updateItem(int id, String newName) async {
    final response = await dioConsumer.put(
      '${EndPoint.yourEndpoint}/$id',
      data: { 'name': newName },
    );
    final data = response.data['data'];
    return YourModel.fromJson(data);
  }
```

Universal Template — DELETE (No meaningful response body)

Universal Template

```
  @override
  Future<void> deleteItem(int id) async {
    await dioConsumer.delete('${EndPoint.yourEndpoint}/$id');
    // Nothing to parse or return. Success = no exception thrown.
  }
```

Feature Examples

Feature Example

```
abstract class ProductRemoteDataSource {
  Future<ProductModel> createProduct(String name, double price);
}

class ProductRemoteDataSourceImpl implements ProductRemoteDataSource {
  final DioClient dioConsumer;
  ProductRemoteDataSourceImpl({ required this.dioConsumer });
```

```
  @override
  Future<ProductModel> createProduct(String name, double price) async {
    final response = await dioConsumer.post(
      EndPoint.createProduct,
      data: {
        'name':  name,  // keys must match the API doc request body
        'price': price,
      },
    );
    final data = response.data['data'];
    return ProductModel.fromJson(data);
  }

  Future<ProductModel> getProduct(int id) async {
    final response = await dioConsumer.get('${EndPoint.products}/$id');
    return ProductModel.fromJson(response.data['data']);
  }

  Future<List<ProductModel>> getProducts() async {
    final response = await dioConsumer.get(EndPoint.products);
    final List<dynamic> list = response.data['data'];
    return ProductModel.fromJsonList(list);
  }

  Future<void> deleteProduct(int id) async {
    await dioConsumer.delete('${EndPoint.products}/$id');
  }
}
```

📌 **NOTE:** Always dig into the correct key of response.data before passing it to fromJson. Check your API doc's success response to know the exact path (e.g. data['data'], data['result'], data['items']).

Step 5 Write the Repository Interface

The Repository Interface is a contract that lives in the domain layer. It declares what actions are available **without** saying anything about how they are performed. This is what your UseCases and tests will depend on. Because it lives in the domain layer, it only knows about Entities — never Models.

Folder: lib/features/[feature]/domain/repositories/

The Either Rule

In standard Dart, you usually throw exceptions when an error occurs. In Clean Architecture, throwing exceptions across layers is dangerous. Instead, you must return an **Either<Failure, T>** from the **dartz package.**

- **The Left Side:** Always carries a **Failure (your custom error object).**
- **The Right Side:** Always carries your **expected data (success).**

This pattern mathematically forces the developer to handle both the success and error states in the UI layer.

Universal Template

Universal Template

```
import 'package:dartz/dartz.dart';
import '../../../../core/error/failures.dart';
import '../entities/your_entity.dart'; // NEVER import a Model here

abstract class FeatureRepository {

  // 1. POST / PUT (Sends data, returns a single entity)
  Future<Either<Failure, YourEntity>> performAction(String inputA, int inputB);

  // 2. GET (Retrieves a single entity by ID)
  Future<Either<Failure, YourEntity>> getSingleItem(int id);

  // 3. GET (Retrieves a list of entities)
  Future<Either<Failure, List<YourEntity>>> getItems();

  // 4. DELETE / Action (No meaningful return data)
  // Right side is void because success simply means no error happened.
  Future<Either<Failure, void>> deleteItem(int id);
}
```

Feature Example

Feature Example

```
import 'package:dartz/dartz.dart';
import '../../../../core/error/failures.dart';
import '../entities/product_entity.dart';

abstract class ProductRepository {
  // Return type pattern: Either<Failure, YourReturnType>
  Future<Either<Failure, ProductEntity>>       createProduct(String name, double price);
  Future<Either<Failure, ProductEntity>>       getProduct(int id);
  Future<Either<Failure, List<ProductEntity>>> getProducts();
  Future<Either<Failure, void>>                deleteProduct(int id);
}
```

📗 **Rule:** Always use Either<Failure, T> from the dartz package. The Left side carries a Failure (error); the Right side carries your data (success). This forces safe error handling throughout your codebase.

Step 6 Implement the Repository

The Repository Implementation lives in the data layer and fulfils the contract defined in Step 5. Its primary job is error handling — it **wraps** every data source call in a **try-catch block** and **converts exceptions** into **typed Failures**.

Folder: lib/features/[feature]/data/repositories/

Key Rules

- **The Only Try-Catch:** This is the absolute only place in your API integration flow where you should write a try-catch block for a network call.
- **The Poly-Trick:** Your Domain Interface (Step 5) asks for an Entity. Your Data Source (Step 4) returns a Model. Because every Model extends its Entity, returning the Model perfectly satisfies the Entity contract.
- **Packaging the Result:** Wrap success in Right(data). Catch exceptions, convert to a Failure, and wrap in Left(Failure).

Universal Template

Universal Template

```
import 'package:dartz/dartz.dart';
import '../../../../core/error/exceptions.dart';
import '../../../../core/error/failures.dart';
import '../../../../core/error/failures_mapper.dart';
import '../datasources/feature_remote_data_source.dart';
import '../../domain/entities/your_entity.dart';
import '../../domain/repositories/feature_repository.dart';

class FeatureRepositoryImpl implements FeatureRepository {
  final FeatureRemoteDataSource remoteDataSource;
  FeatureRepositoryImpl({required this.remoteDataSource});

  // 1. Standard Method Template (Returns Data)
  @override
  Future<Either<Failure, YourEntity>> performAction(String input) async {
    try {
      final model = await remoteDataSource.performAction(input);
      return Right(model);                        // ✅ success
    } on AppException catch (e) {
      return Left(mapExceptionToFailure(e));      // ❌ known API error
    } catch (e) {
      return Left(UnknownFailure(e.toString())); // ❌ unexpected crash
    }
  }

  // 2. Action Method Template (Returns Void / No Data)
  @override
  Future<Either<Failure, void>> deleteItem(int id) async {
    try {
      await remoteDataSource.deleteItem(id);
      return const Right(null);
    } on AppException catch (e) {
      return Left(mapExceptionToFailure(e));
    } catch (e) {
      return Left(UnknownFailure(e.toString()));
```

```
    }
  }
}
```

Feature Example

Feature Example

```
class ProductRepositoryImpl implements ProductRepository {
  final ProductRemoteDataSource remoteDataSource;
  ProductRepositoryImpl({ required this.remoteDataSource });

  @override
  Future<Either<Failure, ProductEntity>> createProduct(
    String name,
    double price,
  ) async {
    try {
      final model = await remoteDataSource.createProduct(name, price);
      return Right(model);
    } on AppException catch (e) {
      return Left(mapExceptionToFailure(e));
    } catch (e) {
      return Left(UnknownFailure(e.toString()));
    }
  }

  @override
  Future<Either<Failure, List<ProductEntity>>> getProducts() async {
    try {
      final models = await remoteDataSource.getProducts();
      return Right(models);
    } on AppException catch (e) {
      return Left(mapExceptionToFailure(e));
    } catch (e) {
      return Left(UnknownFailure(e.toString()));
    }
  }
}
```

Step 7 Create the UseCase

A UseCase is a thin **bridge class** with a single call() method. The UI never talks directly to the Repository — it always goes through a UseCase. This keeps business logic out of your widgets and makes testing straightforward.

Folder: lib/features/[feature]/domain/usecases/

Design Principles

- **Single Responsibility:** One UseCase equals exactly one action. If a screen fetches products and deletes a product, use two separate UseCases — not one giant class.
- **Business Logic Isolation:** If you need to validate input or combine data from two different repositories before showing it to the UI, that logic lives here. This keeps your UI widgets completely free of business logic.
- **The call() Magic:** Always name the main method call. In Dart, if a class has a method named call(), you can invoke the instance directly as if it were a function — e.g. await loginUseCase(email, password). This makes your Cubit code incredibly clean.

Because UseCases live in the Domain layer, they only import Entities and the Repository Interface. They never import Models or Dio.

Universal Template — With Parameters (POST, PUT, or GET by ID)

Universal Template

```
import 'package:dartz/dartz.dart';
import '../../../../core/error/failures.dart';
import '../entities/your_entity.dart';
import '../repositories/feature_repository.dart';

class PerformActionUseCase {
  final FeatureRepository repository;
  PerformActionUseCase({required this.repository});

  Future<Either<Failure, YourEntity>> call(String inputA, int inputB) async {
    return await repository.performAction(inputA, inputB);
  }
}
```

Universal Template — No Parameters (GET all)

Universal Template

```
class GetItemsUseCase {
  final FeatureRepository repository;
  GetItemsUseCase({required this.repository});

  Future<Either<Failure, List<YourEntity>>> call() async {
    return await repository.getItems();
  }
}
```

Feature Example — With Parameters

Feature Example

```
class CreateProductUseCase {
  final ProductRepository repository;
  CreateProductUseCase({ required this.repository });

  Future<Either<Failure, ProductEntity>> call(
    String name,
    double price,
  ) async {
```

```
    return await repository.createProduct(name, price);
  }
}
```

Feature Example — No Parameters

Feature Example

```
class GetProductsUseCase {
  final ProductRepository repository;
  GetProductsUseCase({ required this.repository });

  Future<Either<Failure, List<ProductEntity>>> call() async {
    return await repository.getProducts();
  }
}
```

📌 **NOTE:** Always name the method 'call'. This lets you invoke the UseCase instance directly — e.g. await getProductsUseCase() — which reads cleanly inside the Cubit.

Step 8 Create the States and Cubit

The Cubit **manages UI state**. It calls the UseCase, then emits one of four possible states based on the result. The states file and the Cubit file are kept in the same folder.

Folder: lib/features/[feature]/presentation/manager/

The Four Universal States

Every API call feature relies on exactly four states:

6. **Initial:** The screen just opened; nothing has happened yet.
7. **Loading:** The UseCase was called; the UI should show a spinner.
8. **Success:** The data arrived safely; this state must carry the Entity data so the UI can display it.
9. **Error:** The try-catch in Step 6 caught an issue; this state must carry a friendly String message so the UI can show a Snackbar.

The Fold Rule

When you call your UseCase, it returns an Either<Failure, T>. You must unpack it using result.fold(). The first callback handles the left side (Failure), and the second handles the right side (Success). Never bypass this with .getOrElse.

Universal Template — States (feature_state.dart)

Universal Template

```
import '../../domain/entities/your_entity.dart';

abstract class FeatureState {}
```

```
// 1. App has not triggered any action yet
class FeatureInitial extends FeatureState {}

// 2. API call is in progress - show a spinner
class FeatureLoading extends FeatureState {}

// 3. API call succeeded - carry the data the UI needs
class FeatureSuccess extends FeatureState {
  final YourEntity data; // Or List<YourEntity> for arrays
  FeatureSuccess({required this.data});
}

// 4. API call failed - carry the friendly error message
class FeatureError extends FeatureState {
  final String message;
  FeatureError({required this.message});
}
```

Universal Template — Cubit (feature_cubit.dart)

Universal Template

```
import 'package:flutter_bloc/flutter_bloc.dart';
import '../../domain/usecases/perform_action_usecase.dart';
import 'feature_state.dart';

class FeatureCubit extends Cubit<FeatureState> {
  final PerformActionUseCase performActionUseCase;

  FeatureCubit({
    required this.performActionUseCase,
  }) : super(FeatureInitial()); // Always start with the Initial state

  Future<void> performAction(String inputA, int inputB) async {
    emit(FeatureLoading());
    final result = await performActionUseCase(inputA, inputB);
    result.fold(
      (failure) => emit(FeatureError(message: failure.message)),
      (data)    => emit(FeatureSuccess(data: data)),
    );
  }
}
```

Feature Example — States

Feature Example

```
import '../../domain/entities/product_entity.dart';

abstract class ProductState {}

class ProductInitial     extends ProductState {}
```

```
class ProductLoading     extends ProductState {}

class ProductSuccess extends ProductState {
  final ProductEntity product;
  ProductSuccess({ required this.product });
}

class ProductListSuccess extends ProductState {
  final List<ProductEntity> products;
  ProductListSuccess({ required this.products });
}

class ProductError extends ProductState {
  final String message;
  ProductError({ required this.message });
}
```

Feature Example — Cubit

Feature Example

```
import 'package:flutter_bloc/flutter_bloc.dart';
import '../../domain/usecases/create_product_usecase.dart';
import '../../domain/usecases/get_products_usecase.dart';
import 'product_state.dart';

class ProductCubit extends Cubit<ProductState> {
  final CreateProductUseCase createProductUseCase;
  final GetProductsUseCase   getProductsUseCase;

  ProductCubit({
    required this.createProductUseCase,
    required this.getProductsUseCase,
  }) : super(ProductInitial());

  Future<void> createProduct(String name, double price) async {
    emit(ProductLoading());
    final result = await createProductUseCase(name, price);
    result.fold(
      (failure) => emit(ProductError(message: failure.message)),
      (product) => emit(ProductSuccess(product: product)),
    );
  }

  Future<void> getProducts() async {
    emit(ProductLoading());
    final result = await getProductsUseCase();
    result.fold(
      (failure) => emit(ProductError(message: failure.message)),
      (products) => emit(ProductListSuccess(products: products)),
    );
  }
}
```

🔑 **Key Rule:** Use result.fold() to unpack the Either. The first callback receives a Failure (Left) and the second receives your data (Right). Never access result.getOrElse or throw — always fold.

Step 9 Dependency Injection

Use **get_it** as a service locator to **wire** all classes together. Think of it as a master tool shed for your app — all tools are built exactly once when the app starts and stored for retrieval anywhere using sl<ClassName>().

File: lib/core/di/service_locator.dart

The Registration Order (Strict Rule)

You cannot register a UseCase **before you register its Repository**, because the UseCase needs the Repository to exist first. Always build from the inside out:

1. Core (Dio, Cache)
2. Data Sources
3. Repositories
4. UseCases
5. Cubits

Factory vs. Singleton — The Cubit Trap

- **registerLazySingleton:** Creates exactly ONE instance of a class and keeps it alive forever. Use this for network clients, data sources, repositories, and UseCases.
- **registerFactory:** Creates a brand new instance every single time you ask for it. **You MUST use this for Cubits**. If you make a Cubit a singleton, its state will persist even after the user closes the screen, causing stale data bugs when they reopen it.

Universal Template

Universal Template

```
import 'package:get_it/get_it.dart';
import 'package:dio/dio.dart';
// ... import your feature files here ...

final sl = GetIt.instance;

void setupServiceLocator() {
  // 1. Core Infrastructure (Singletons)
  sl.registerLazySingleton<Dio>(() => Dio());
  sl.registerLazySingleton<DioClient>(() => DioClient(dio: sl()));

  // FEATURE: [Feature Name]
  // 2. Data Sources
  sl.registerLazySingleton<FeatureRemoteDataSource>(
    () => FeatureRemoteDataSourceImpl(dioConsumer: sl()),
```

```
  );

  // 3. Repositories (Interface mapped to Implementation)
  sl.registerLazySingleton<FeatureRepository>(
    () => FeatureRepositoryImpl(remoteDataSource: sl()),
  );

  // 4. UseCases
  sl.registerLazySingleton(() => PerformActionUseCase(repository: sl()));
  sl.registerLazySingleton(() => GetItemsUseCase(repository: sl()));

  // 5. Cubits (ALWAYS Factory!)
  sl.registerFactory(() => FeatureCubit(
    performActionUseCase: sl(),
    getItemsUseCase:      sl(),
  ));
}
```

Feature Example

Feature Example

```
// 1. Core infrastructure
sl.registerLazySingleton<Dio>(() => Dio());
sl.registerLazySingleton<CacheHelper>(() => CacheHelper());
sl.registerLazySingleton<DioClient>(() => DioClient(cacheHelper: sl()));

// 2. Data Sources
sl.registerLazySingleton<ProductRemoteDataSource>(
  () => ProductRemoteDataSourceImpl(dioConsumer: sl()),
);

// 3. Repositories
sl.registerLazySingleton<ProductRepository>(
  () => ProductRepositoryImpl(remoteDataSource: sl()),
);

// 4. UseCases
sl.registerLazySingleton(() => CreateProductUseCase(repository: sl()));
sl.registerLazySingleton(() => GetProductsUseCase(repository: sl()));

// 5. Cubits - ALWAYS use registerFactory
sl.registerFactory(() => ProductCubit(
  createProductUseCase: sl(),
  getProductsUseCase:   sl(),
));
```

⚠️ **Important:** Never register a Cubit as registerLazySingleton. Always use registerFactory. A singleton Cubit persists its state across all screens that use it, causing stale data bugs that are very hard to trace.

Step 10 Connect the UI

Wrap your screen in a BlocProvider to inject the Cubit, and use BlocConsumer to listen for one-time side effects (navigation, Snackbars) and rebuild the UI when state changes.

Folder: lib/features/[feature]/presentation/views/

Listener vs. Builder — The Golden Rule

- **listener (Side Effects):** Runs exactly once per state change. Use this for things that happen in the background: navigating to a new screen, showing a Snackbar, or triggering a dialog. Never return a widget here.
- **builder (UI Drawing):** Runs every single time the state changes. Use this strictly to draw the UI (spinners, lists, text). Never put navigation or Snackbars inside the builder.

Universal Template

Universal Template

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import '../../manager/feature_cubit.dart';
import '../../manager/feature_state.dart';
import '../../../../../core/di/service_locator.dart';

class FeatureScreen extends StatelessWidget {
  const FeatureScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      create: (_) => sl<FeatureCubit>()..getItems(),
      child: Scaffold(
        appBar: AppBar(title: const Text('Your Feature')),
        body: BlocConsumer<FeatureCubit, FeatureState>(
          // LISTENER: One-time side effects
          listener: (context, state) {
            if (state is FeatureError) {
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                  content: Text(state.message),
                  backgroundColor: Colors.red,
                ),
              );
            }
            if (state is FeatureSuccess) {
              // e.g., Navigator.pushReplacementNamed(...)
            }
          },
          // BUILDER: Rebuilds the UI widgets
          builder: (context, state) {
            if (state is FeatureLoading) {
              return const Center(child: CircularProgressIndicator());
```

```
            }
            if (state is FeatureSuccess) {
              return ListView.builder(
                itemCount: state.data.length,
                itemBuilder: (context, i) => Text(state.data[i].name),
              );
            }



            // FeatureInitial or FeatureError - default/empty UI
            return const SizedBox.shrink();
          },
        ),
      ),
    );
  }
}
```

Feature Example

Feature Example

```
class ProductScreen extends StatelessWidget {
  const ProductScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      create: (_) => sl<ProductCubit>()..getProducts(),
      child: BlocConsumer<ProductCubit, ProductState>(
        listener: (context, state) {
          if (state is ProductError) {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(
                content: Text(state.message),
                backgroundColor: Colors.red,
              ),
            );
          }
          if (state is ProductSuccess) {
            // Navigate, show confirmation, etc.
          }
        },
        builder: (context, state) {
          if (state is ProductLoading) {
            return const Center(child: CircularProgressIndicator());
          }
          if (state is ProductListSuccess) {
            return ListView.builder(
              itemCount: state.products.length,
              itemBuilder: (_, i) => ListTile(
                title: Text(state.products[i].name),
              ),
            );
          }
```

```
          return const SizedBox.shrink();
        },
      ),
    );
  }
}
```

📗 **Rule:** Listener is for side effects (things that happen once). Builder is for drawing widgets (things that rebuild on every state). Never put navigation inside the builder.

Variant Endpoint Patterns

The 10-step blueprint handles the most common case — a POST that returns a single object. Below are the adjustments for the two other common patterns.

Pattern A — GET Endpoint That Returns a List

The only steps that differ from the main blueprint are Step 3, Step 4, Step 5, Step 8, and the state type:

- **Step 3 (Model):** Add the static fromJsonList helper.
- **Step 4 (Data Source):** Use dioConsumer.get() with no data body. Parse response.data['data'] as List<dynamic> and call fromJsonList.
- **Step 5 (Repository Interface):** The return type becomes Either<Failure, List<YourEntity>>.
- **Step 8 (States):** Add a separate Success state that carries a List<YourEntity> instead of a single entity.
- **Step 10 (UI):** Check for the list success state in the builder and render a ListView.

Pattern B — DELETE / Action Endpoint with No Meaningful Response Body

Some endpoints (DELETE, logout, mark-as-read, etc.) return only a status code with no data. The adjustments are:

- **Step 2 (Entity):** Skip entirely — there is no data to model.
- **Step 3 (Model):** Skip entirely.
- **Step 4 (Data Source):** Call dioConsumer.delete() and return void. Success means no exception was thrown.
- **Step 5 (Repository Interface):** Use Either<Failure, void> as the return type.
- **Step 6 (Repository Impl):** Return Right(null) on success.
- **Step 7 (UseCase):** Return type is Either<Failure, void>.
- **Step 8 (States):** Your Success state carries no data — class DeleteSuccess extends YourState {}

Pattern C — Merging Shared Entities Across Multiple Endpoints

Sometimes two different API endpoints both return a JSON response that maps to the **same Entity class** — but each response includes slightly different fields. For example, **/auth/login** and **/auth/register** both return a **user** JSON response that maps to **UserEntity** — but login returns **roleId** while register returns **email**. Instead of creating two separate Entity and Model classes, you keep one of each and mark any field that isn't guaranteed by every endpoint as nullable (**Type?**).

Merge or Create New?

- **Merge** when two endpoints return a JSON response that maps to the same Entity class and they share at least **one common** field (like **id**). Only a few fields should differ between them.
- **Create a new Entity** when two endpoints return responses with completely different sets of fields, even if the class name is the same.

> 📗 **Quick Test:** If merging would force more than half of the fields to be nullable, keep them in separate classes.

Pattern C — Implementation Steps:

Step 2 Entity: (✏️ Edit existing files):
Keep the existing Entity class. For every field, ask: **"Does every endpoint that uses this Entity return this field?"** If yes → keep **required**. If no → make it **nullable** (**Type?**) and remove **required** from the constructor.

```
class UserEntity {
  final int     id;                  // required - both endpoints return it
  final String? name;                // nullable - login only
  final int?    roleId;              // nullable - login only
  final String? email;               // nullable - register only
  final List<String>? roles;        // nullable List - login only (needs List Guard in
fromJson)

  const UserEntity({
    required this.id,
    this.name,
    this.roleId,
    this.email,
    this.roles,
  });
}
```

Step 3: Model (✏️ Edit existing files)
Update your existing Model to match the new Entity. Remove **required** from any super fields that are now nullable. Your single fromJson method will now handle the JSON from all endpoints.

> ⚠️ **The List Guard Rule (CRITICAL):** If a JSON key is missing, Dart automatically assigns null to primitive types (like Strings or ints). However, if you have a nullable **List**, you *must* explicitly check for null before converting it. Calling List.from(null) will instantly crash your app.

```
// 1. Constructor: Remove 'required' from nullable super fields
const UserModel({
  required super.id,
  super.name,           // no 'required' - login sends it, register does NOT
  super.roleId,          // no 'required' - login sends it, register does NOT
```

```
  super.email,           // no 'required' - register sends it, login does NOT
  super.roles,           // no 'required' - login sends it, register does NOT
});

// 2. fromJson: One method handles both endpoints
factory UserModel.fromJson(Map<String, dynamic> json) {
  return UserModel(
    id: json['id'],
    name: json['name'], // primitive - automatically null if absent, no guard needed

    // THE LIST GUARD - required for every nullable List field!
    roles: json['roles'] != null
        ? List<String>.from(json['roles'])
        : null,
  );
}
```

Steps 4, 5 & 6: Data Source, Repo Interface, Repo Impl (✏️ Edit existing files)
Do not create new files. You simply add the new endpoint's method directly below the existing method in all three files. The try/catch error handling and Dio network patterns remain exactly the same

```
// Step 4 & 5 (Interface): Add the new signature
Future<AuthEntity> register(String email, String password);

// Step 4 (Data Source Impl): Delegate to the same Model
final response = await dioConsumer.post(EndPoint.register, data: {'email': email, 'password':
password});
return AuthModel.fromJson(response.data['data']);

// Step 6 (Repository Impl): Standard try/catch pattern
final model = await remoteDataSource.register(email, password);
return Right(model);
```

Step 7: UseCase (+ New file)
Always create a new file for a new action. Create your new UseCase (e.g., RegisterUseCase) and inject the exact same shared Repository into it.

Step 8: States & Cubit (✏️ Edit existing files)

- **States:** Add your new specific states (like RegisterSuccess and RegisterError) to the very *bottom* of your existing state file. Never modify or delete the old Login states, as other screens are relying on them.
- **Cubit:** Inject your new UseCase via the constructor, and add your new method alongside the old one.

```
// In the States file (append to bottom):
class RegisterSuccess extends AuthState {
  final AuthEntity auth;
  RegisterSuccess({required this.auth});
}

// In the Cubit file:
// 1. Inject the new UseCase
AuthCubit({ required this.loginUseCase, required this.registerUseCase, ... });

// 2. Add the new method
result.fold(
  (failure) => emit(RegisterError(message: failure.message)),
  (auth)    => emit(RegisterSuccess(auth: auth)),
);
```

Step 9: Dependency Injection (✏️ Edit existing file)

Your Data Source and Repository are already registered from the first endpoint, so leave them alone. You only need to register your new UseCase and update the Cubit to accept it.

```
// 1. Register the new UseCase
sl.registerLazySingleton(() => RegisterUseCase(repository: sl()));

// 2. Update the existing Cubit factory
sl.registerFactory(() => AuthCubit(
  loginUseCase: sl(),
  registerUseCase: sl(),  // ← Just add this new line
));
```

Step 10: UI Screen (+ New file)

Create a brand new screen file for the new action. It will use the exact same shared Cubit via BlocProvider + sl(). In your BlocConsumer, only listen for the specific states this new screen cares about (e.g., ignore LoginSuccess on the Register screen).

> ⚠️ **Nullable Field Guard Rule (CRITICAL):** Because you merged your Entities, many fields are now nullable. You must guard these fields in your UI before trying to render them. You (the developer) know which endpoint triggered the success state, so only attempt to read fields you know that specific endpoint returned.

Handling Nullable Fields in the UI (Step 10):

- **Booleans:** Use field == true (instead of if (field)).
- **Lists:** Use field ?? [].
- **Strings:** Use field ?? 'Default Text'.

```
// Inside your BlocConsumer listener:
if (state is RegisterSuccess) {
  // Safe to read auth.user.id and auth.user.email here!
  Navigator.pushReplacementNamed(context, '/home');
}
```

File-by-File Reference — Adding a Second Endpoint to an Existing Feature

File	Action	What to do
entity.dart (shared)	✏️ Edit	Add new nullable fields for the new endpoint. Keep only id (or the shared key) as required.
model.dart (shared)	✏️ Edit	Remove required from nullable super fields. In fromJson, guard any nullable List with a null check before calling List.from().
remote_data_source.dart	✏️ Edit	Add the new method to both the abstract class and the implementation.
repository.dart (interface)	✏️ Edit	Add the new method signature. Return type reuses the same Entity.
repository_impl.dart	✏️ Edit	Add the new method with the same try/catch pattern. Delegate to the data source.
new_action_usecase.dart	+ New file	One UseCase per action — always a new file. Inject the same Repository.

File	Action	What to do
feature_state.dart	✏️ Edit	Append NewActionSuccess and NewActionError states at the bottom. Never touch existing states.
feature_cubit.dart	✏️ Edit	Inject the new UseCase via the constructor. Add the new method alongside existing ones.
service_locator.dart	✏️ Edit	Register the new UseCase with registerLazySingleton. Add it to the Cubit's registerFactory. DataSource and Repository are already registered — leave them.
new_screen.dart	+ New file	New screen for the new action. Uses the same Cubit via BlocProvider + sl(). Only listen for the states this screen cares about.

Quick Reference — Checklist per Endpoint

When you receive a new API document, work through this checklist in order. Check each item off before moving to the next.

Step	What to Produce
Step 1: Endpoint URL	Add path string to end_points.dart
Step 2: Entity	Pure Dart class — fields from API response body
Step 3: Model	Extends Entity, adds fromJson (+ fromJsonList for arrays)
Step 4: Remote Data Source	Dio call (GET / POST / PUT / DELETE) + JSON parsing
Step 5: Repository Interface	Abstract contract with Either<Failure, T> return types
Step 6: Repository Implementation	try/catch wrapping, Right() or Left(Failure)
Step 7: UseCase	Thin bridge with a call() method
Step 8: States + Cubit	4 states: Initial, Loading, Success, Error
Step 9: Dependency Injection	Register Data Source, Repository, UseCase, Cubit (Factory)
Step 10: UI Screen	BlocProvider + BlocConsumer (listener + builder)

Folder Structure Reference

Every feature follows the same directory layout inside lib/features/[feature_name]/:

```
lib/
├── core/
|   ├── api/
|   |   ├── end_points.dart          ← Step 1
|   |   └── dio_client.dart
|   ├── di/
|   |   └── service_locator.dart     ← Step 9
|   └── error/
|       ├── exceptions.dart
|       ├── failures.dart
|       └── failures_mapper.dart
|
└── features/
```

```
    └── [feature_name]/
        ├── domain/                  ← Pure Dart, no Flutter/Dio dependencies
        |   ├── entities/            ← Step 2
        |   ├── repositories/        ← Step 5 (interface)
        |   └── usecases/            ← Step 7
        |
        ├── data/                    ← Implements domain contracts
        |   ├── models/              ← Step 3
        |   ├── datasources/         ← Step 4
        |   └── repositories/        ← Step 6 (implementation)
        |
        └── presentation/            ← Flutter UI
            ├── manager/             ← Step 8 (cubit + states)
            └── views/               ← Step 10
```

Golden Rules

- **One UseCase per action.** GetProducts and CreateProduct are two separate UseCase classes, not one class with two methods.
- **Domain never imports from data or presentation.** The entities/ and repositories/ (interface) folders must never contain an import of Dio, Cubit, or any model.
- **Register Cubits with registerFactory.** All other classes can use registerLazySingleton.
- **fromJson keys must match the API doc exactly.** A wrong key silently produces null instead of a crash — always double-check.
- **fold() every Either.** Never access .getOrElse on an untested Either. Always fold and handle both sides.
- **Listener is for side effects; builder is for widgets.** Never navigate inside the builder.
- **One Entity and Model per JSON map shape, not per endpoint.** When two endpoints return a JSON map that feeds into the same Entity class, keep one Entity and one Model. Make the fields that only some endpoints return nullable (Type?). Only create a separate Entity class when the two JSON maps carry genuinely different sets of fields. In your widget tree, guard nullable `bool?` fields with `field == true` and nullable `List?` fields with `field ?? []`. In fromJson, guard any key that may be absent before calling `List.from()` to prevent a runtime crash.